

Assignment 13: Intro Seaborn - P.2

Name Ayleen Santander

Part 1: Solve the task below. (NOTE: Download bigMart.csv and save it to your file directory.)

1. Import libraries

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

```
!pip install fastcluster
```

```
Collecting fastcluster
  Downloading fastcluster-1.3.0-cp312-cp312-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.7 kB)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from fastcluster) (2.0.2)
Downloading fastcluster-1.3.0-cp312-cp312-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
----- 194.1/194.1 kB 8.5 MB/s eta 0:00:00

Installing collected packages: fastcluster
Successfully installed fastcluster-1.3.0
```

```
#Set pandas diplay format to show 2 decimal places for floating- point numbers/ when I apply this format will apply for entire notebook
pd.options.display.float_format = '{:.2f}'.format
```

2. Load and cleaning dataset

```
b_m = pd.read_csv('bigMart.csv')
print(b_m.head())
print(b_m.describe().head())
print(b_m.info())

sns.set(style="darkgrid")
```

```
Item_Identifier  Item_Weight  Item_Fat_Content  Item_Visibility  \
0      FDA15          9.30          Low Fat          0.02
1      DRC01          5.92          Regular          0.02
2      FDN15         17.50          Low Fat          0.02
3      FDX07         19.20          Regular          0.00
4      NCD19          8.93          Low Fat          0.00

Item_Type  Item_MRP  Outlet_Identifier  \
0      Dairy      249.81          OUT049
1  Soft Drinks      48.27          OUT018
2      Meat     141.62          OUT049
3  Fruits and Vegetables  182.09          OUT010
4      Household      53.86          OUT013

Outlet_Establishment_Year  Outlet_Size  Outlet_Location_Type  \
0              1999          Medium          Tier 1
1              2009          Medium          Tier 3
2              1999          Medium          Tier 1
3              1998             NaN          Tier 3
4              1987             High          Tier 3

Outlet_Type  Item_Outlet_Sales
0  Supermarket Type1          3735.14
1  Supermarket Type2          443.42
2  Supermarket Type1         2097.27
3   Grocery Store          732.38
4  Supermarket Type1          994.71

Item_Weight  Item_Visibility  Item_MRP  Outlet_Establishment_Year  \
count      7060.00          8523.00      8523.00          8523.00
mean         12.86           0.07       140.99          1997.83
std          4.64           0.05        62.28           8.37
min          4.55           0.00        31.29          1985.00
25%          8.77           0.03        93.83          1987.00

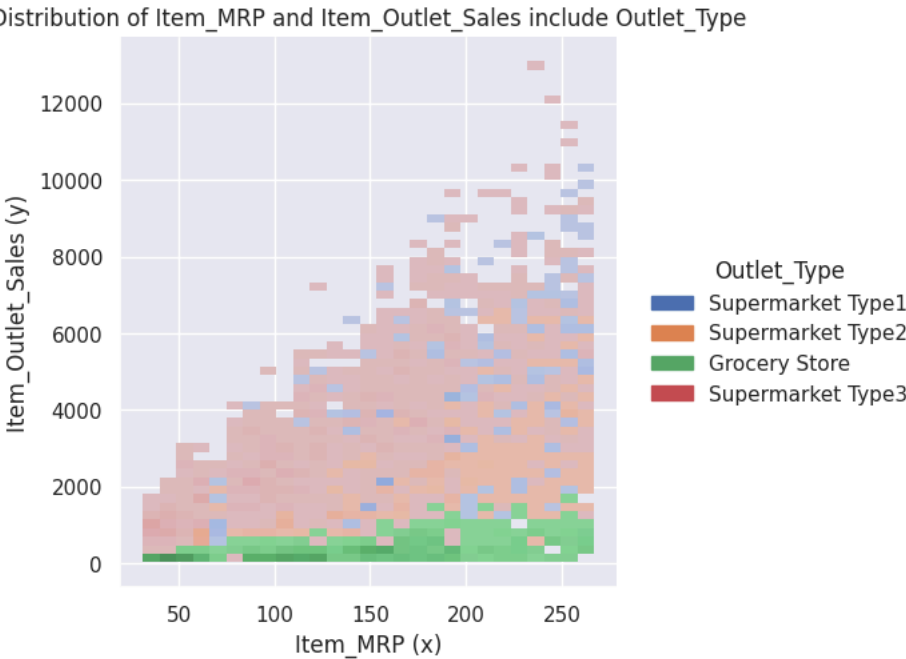
Item_Outlet_Sales
count      8523.00
mean       2181.29
std       1706.50
min        33.29
25%        834.25
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Item_Identifier        8523 non-null  object
 1   Item_Weight            7060 non-null  float64
 2   Item_Fat_Content       8523 non-null  object
 3   Item_Visibility        8523 non-null  float64
 4   Item_Type              8523 non-null  object
 5   Item_MRP               8523 non-null  float64
 6   Outlet_Identifier      8523 non-null  object
 7   Outlet_Establishment_Year  8523 non-null  int64
 8   Outlet_Size            6113 non-null  object
 9   Outlet_Location_Type   8523 non-null  object
10   Outlet_Type            8523 non-null  object
11   Item_Outlet_Sales      8523 non-null  float64
dtypes: float64(4), int64(1), object(7)
```

Part 2: Visualization Tasks: (NOTE: For each task, customize plots by adding a title, x-label, and y-label)

1. Displot (x, y):

- Create a distribution plot (displot) showing the distribution of Item_MRP (x) and Item_Outlet_Sales (y).
- Parameters to explore: hue.
- Note:
 - Item_MRP = Maximum Retail Price for a Product
 - Item_Outlet_Sales = Sales of the product In the Specific outlet
 - Outlet_Type = Grocery store or a type of supermarket
- Plot (1) Analysis: This scatter plot shows the distribution of Maximum Retail Price (MRP) against item outlet sales, with points colored by store type (grocery store vs. supermarket). There is a clear concentration of products on the higher-MRP side (170—250), and overall MRP shows a positive association with sales as price increases, sales tend to increase as well, especially in supermarkets. Supermarkets dominate the higher sales values (1600—8000) across all price ranges. Grocery stores, by contrast, stay at consistently lower sales (0—1500) levels even for higher-priced items. A possible explanation could be lower product variety or limited stock in grocery stores, though this chart alone doesn't confirm that it would need to be checked against the actual item count or inventory data.

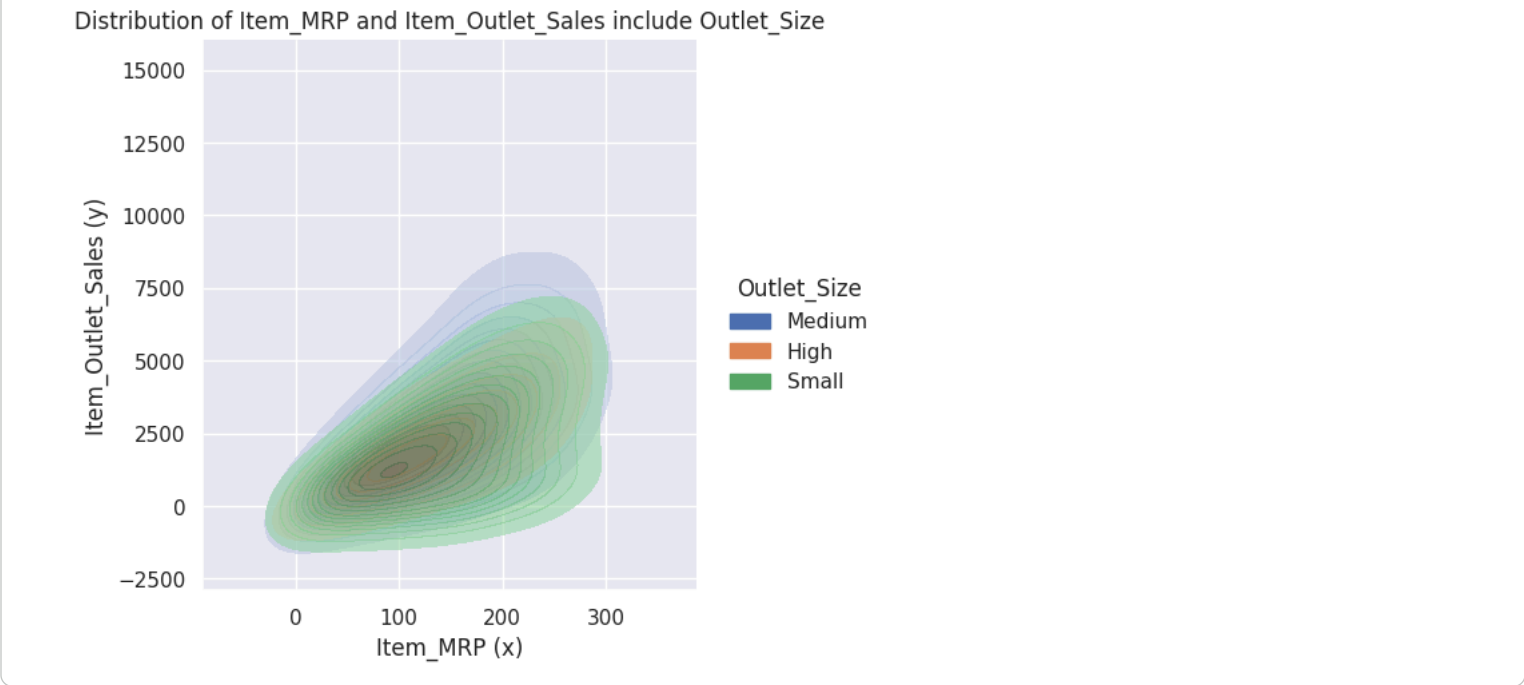
```
# Displot (x,y)
sns.displot(b_m, x= 'Item_MRP', y='Item_Outlet_Sales',hue="Outlet_Type")
plt.title('Distribution of Item_MRP and Item_Outlet_Sales include Outlet_Type')
plt.xlabel('Item_MRP (x)')
plt.ylabel('Item_Outlet_Sales (y)')
plt.show()
```



2. Displot(x, y) with kind=kde:

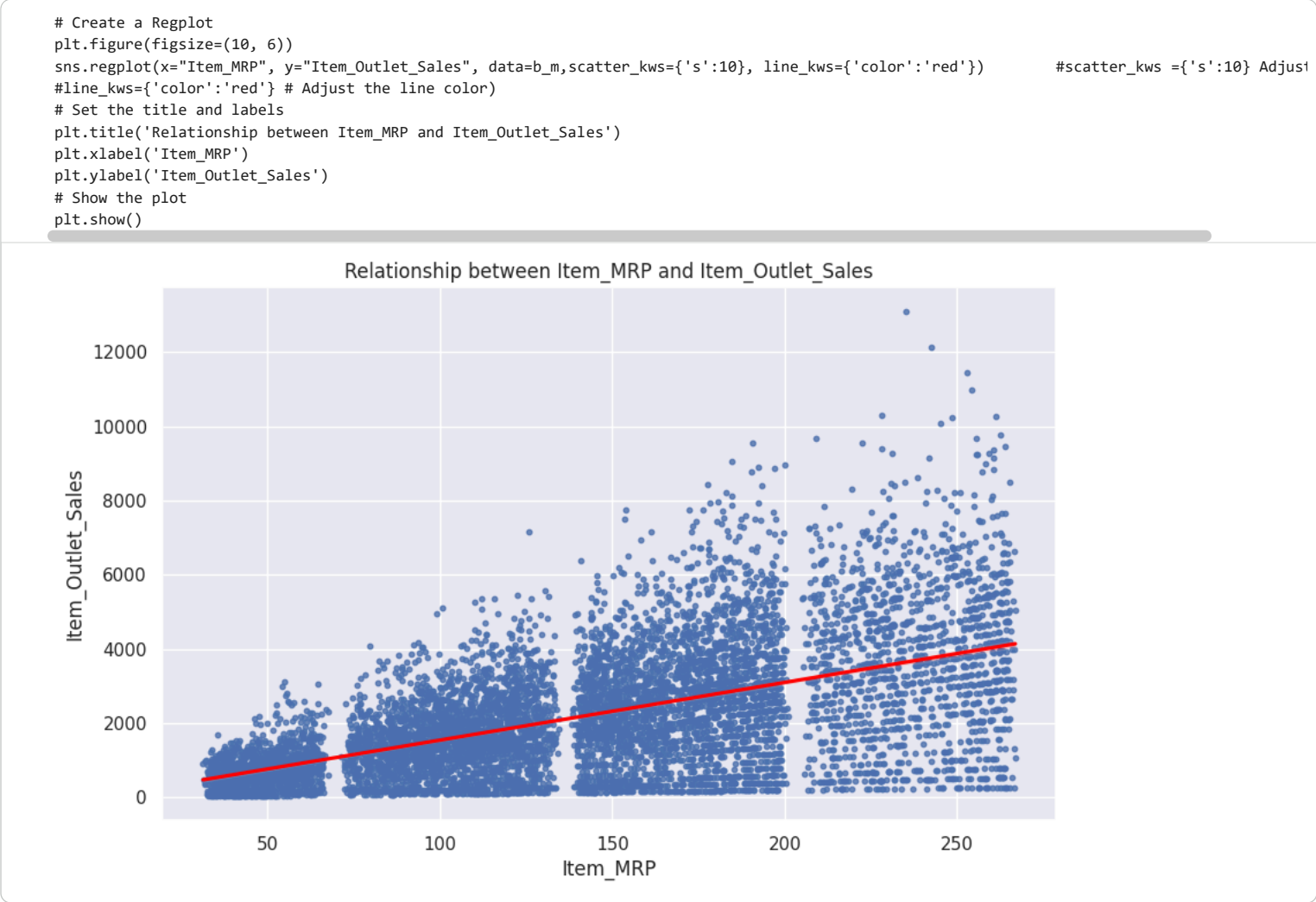
- Create a distribution plot (displot) showing the distribution of Item_MRP (x) and Item_Outlet_Sales (y) with kernel density estimation (KDE).
- Parameters to explore: hue, kde. -Note:
 - Outlet_Size = The size of the store (Area Size Category) CODE: -levels=15 = more contour bands = finer detail in dense zones - thresh=0.05 =lower = shows more of the spread; higher = only the densest core
 - alpha=0.6 = overlapping hue groups stay visible
- Plot (2) Explanation: Now I evaluate if the size of the store (area size category) present any relationship with Item MRP and Item Outlet sales and with is the best combination between them. The High - Density Peak, the most prodcuts cluster around MRP (60)*vs*Sales(500) is when the Outlet Size = High. Smooth Gradients = shows MRP and Sales move togehter, its means higher MRP tends to come with higher Sales = positive relationship. Spread = shows consisten and predictable behavior (rings analysis). Compare with Outlet size shows medium size MRP around (250)*withSalespeak*8500, and small size Sales lower around (6500)*andMRP*(250) not represent big different considerate medium vs small those ones represent importa sales levels and high size Sales below (6000)*andMRP*(290) ish.

```
# Displot with kind="kde" and 'hue'
sns.displot(b_m, x= 'Item_MRP', y='Item_Outlet_Sales',hue="Outlet_Size", kind='kde', fill = True, levels=15, thresh=0.05, alpha=0.5, bw_adjust
plt.title('Distribution of Item_MRP and Item_Outlet_Sales include Outlet_Size')
plt.xlabel('Item_MRP (x)')
plt.ylabel('Item_Outlet_Sales (y)')
plt.show()
```



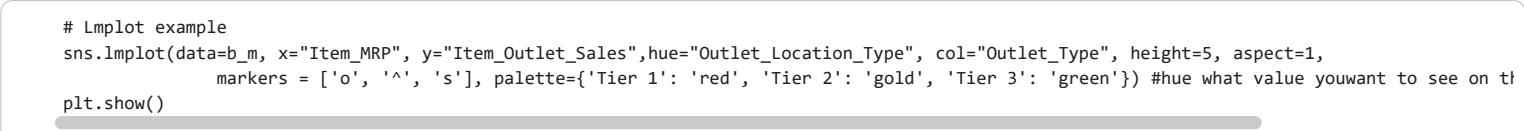
3. Regplot():

- Create a scatterplot with a regression line showing the relationship between Item_MRP and Item_Outlet_Sales.
 - Parameters to explore: scatter_kws, line_kws,
- Plot (3) Explanation = The regression line slopes upward, shows a positive relationship between MRP and Sales, explain higher item priced tend to generate higher sales.



4. Lmplot():

- Create a scatterplot with a regression line showing the relationship between Item_MRP and Item_Outlet_Sales, with different colors representing different Outlet_Type.
 - Parameters to explore: hue, markers, palette.
 - NOTE: Markets should be a set of 4 strings e.g. markers = ['o', 'x', '^', 's']
 - Note:
 - Outlet_Type = Grocery store or a type of supermarket
 - Outlet_Location_Type = In Terms of city Tiers (Size) CODE = - palette={'Tier 1': 'red', 'Tier 2': 'gold', 'Tier 3': 'green'}= match actual tier labels
- Plot (4) Explanation =



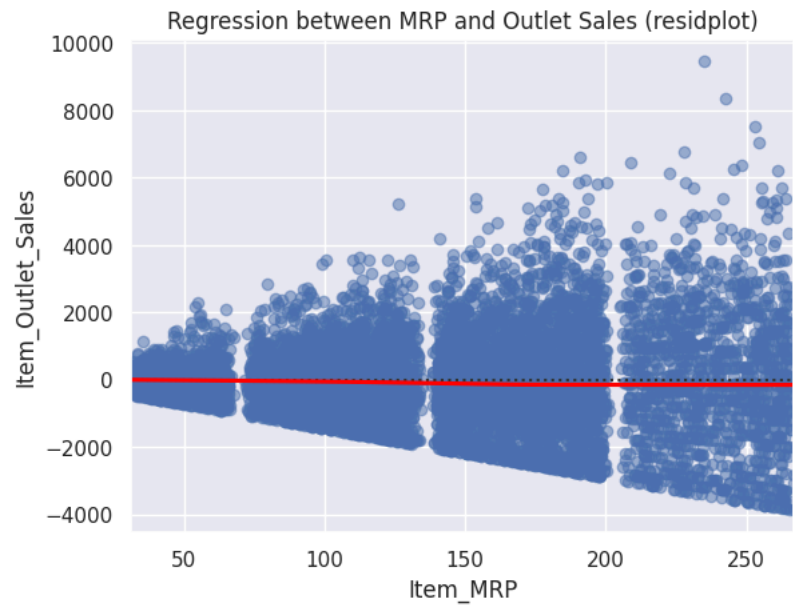


5. Residplot():

- Create a residual plot showing the residuals of the regression between Item_MRP and Item_Outlet_Sales.
- Parameters to explore: lowess, scatter_kws.

-Plot (5) Explanation : The regression lines show the model's linear formula predicted the Sales perfectly, y =0. The residual plot (y) shows a clear cone shape, are small at low MRP and spread widely as MRP increases, it means while increase the MPR prices good relationship with Sales.

```
# Residplot
sns.residplot(data=b_m, x="Item_MRP", y="Item_Outlet_Sales",lowess=True, scatter_kws={"alpha": 0.5},line_kws={"color": "red"})
plt.title('Regression between MRP and Outlet Sales (residplot)')
plt.xlabel('Item_MRP')
plt.ylabel('Item_Outlet_Sales')
plt.show()
```



- Create a pivot table to use for the heatmap and clustermap:

1. Item_Outlet_Sales for values; 2. Item_Fat_Content for index;3. Outlet_Type for columns; 4. Sum function for aggfunc

6. Heatmap():

- Create a heatmap showing the correlation between different numerical variables in the dataset.
- Parameters to explore: annot, cmap.

```
clone_b_m = b_m.copy()
```

```
# Create a pivot table for the heatmap and clustermap
b_m.columns = b_m.columns.str.strip()

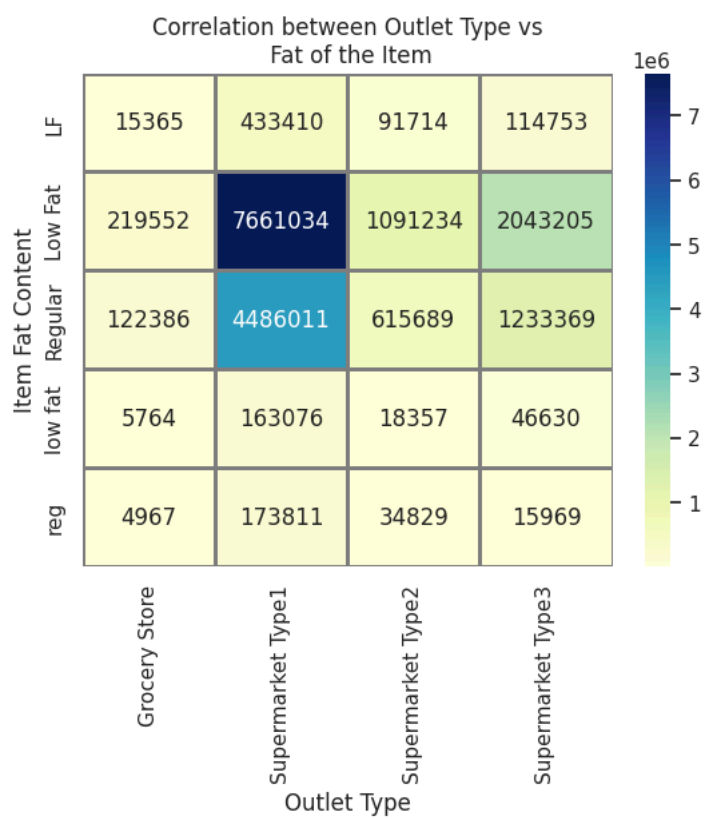
pivot_clone_b_m = pd.pivot_table(
    clone_b_m,
    values='Item_Outlet_Sales',
    index='Item_Fat_Content',
    columns='Outlet_Type',
    aggfunc='sum'
)
print(pivot_clone_b_m)
```

Outlet_Type	Grocery Store	Supermarket Type1	Supermarket Type2	\
Item_Fat_Content				
LF	15365.33	433410.50	91713.95	
Low Fat	219552.21	7661034.36	1091233.55	
Regular	122386.02	4486010.59	615688.56	
low fat	5763.83	163075.73	18357.44	
reg	4966.87	173811.08	34829.33	
Outlet_Type	Supermarket Type3			

Item_Fat_Content	114752.63
LF	2043205.04
Low Fat	1233369.20
Regular	46629.97
low fat	15969.21
reg	

plot (6) Explanation = this plot shows the correlation between the variables Outlet Type and Item Fat Content, the plot immediatly shows in 'darkblue' color has a strong correlation = 7 score, in the Supermarket type 1 with Low fat, it means customer in that area prefer to buy product contain lower fat and the same supermarket also has medium correlation with product contain regular fat, also in this point customer also prefer regular food, one of the lower correlation is Grocery vs reg lower correlation, means the sales in this area are lower so customer are not very attractive with this kind products.

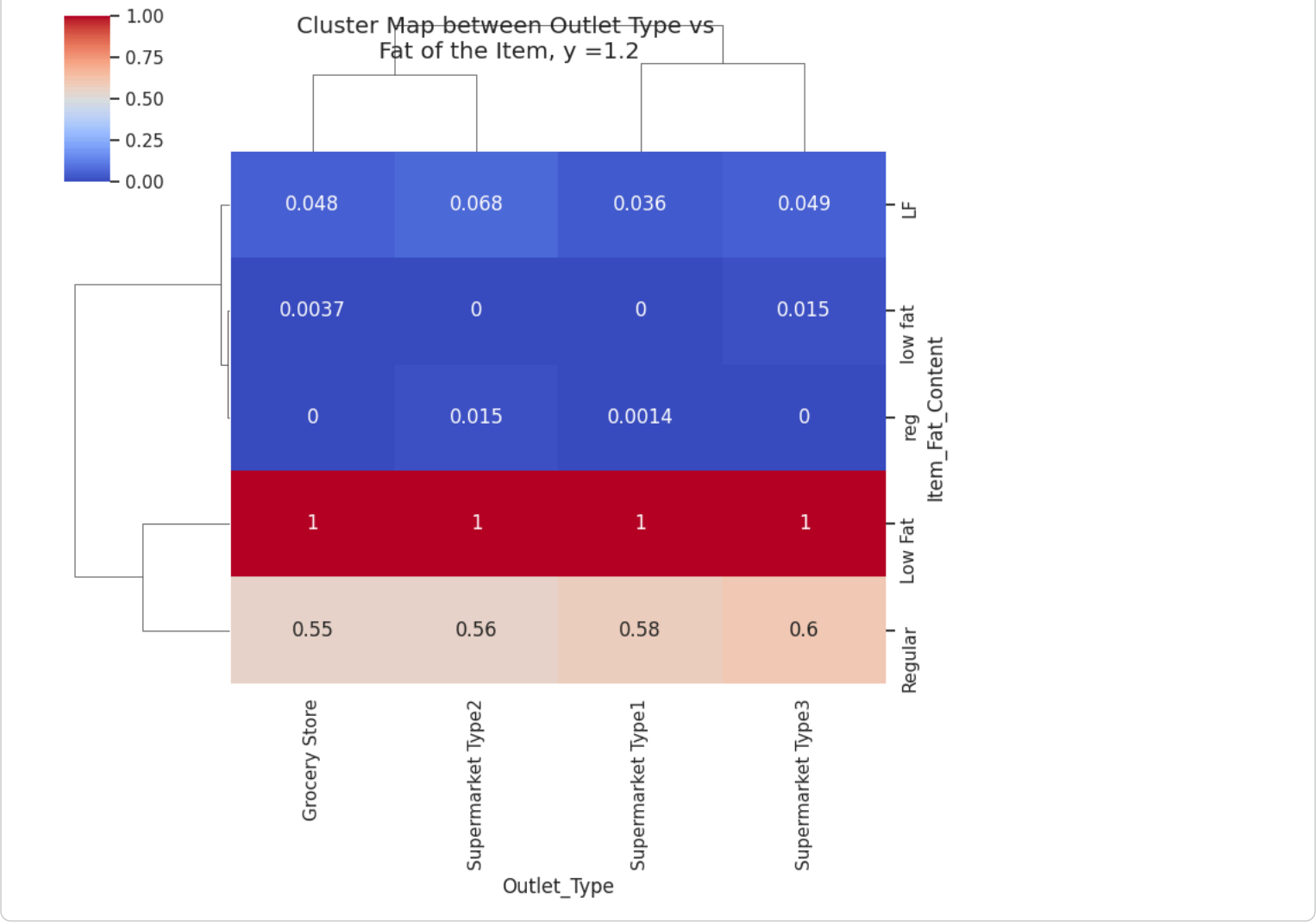
```
from matplotlib.textpath import text_to_path
sns.heatmap(pivot_clone_b_m, annot=True, fmt=".0f", cmap="YlGnBu", linewidths=1, linecolor='gray')
plt.title('Correlation between Outlet Type vs\n Fat of the Item')
plt.xlabel('Outlet Type')
plt.ylabel('Item Fat Content')
plt.show()
```



7. Clustermap():

- Create a clustermap showing the hierarchical clustering of the correlation between different numerical variables in the dataset.
- Parameters to explore: annot, cmap, figsize.

```
#Clean NaNs before clustering
pivot_clean = pivot_clone_b_m.fillna(0)
#Create a Cluster plot
sns.clustermap(pivot_clean, cmap="coolwarm", standard_scale=1,annot=True, figsize=(8, 8))
plt.suptitle('Cluster Map between Outlet Type vs\n Fat of the Item, y =1.2')
plt.show()
```



8. FacetGrid (using FacetGrid.map_dataframe):

- Create a FacetGrid with rows representing different Outlet_Size and columns representing different Outlet_Location_Type, plotting the relationship between Item_MRP and Item_Outlet_Sales.
- Parameters to explore: col, hue, palette. Add legend and save the figure as an image.

```
# Build the grid: rows = Outlet_Size, columns = Outlet_Location_Type
g = sns.FacetGrid(clone_b_m, row="Outlet_Size", col="Outlet_Location_Type", hue="Outlet_Location_Type", # color by location tier
                  palette="Set2", height=4, aspect=1.2, margin_titles=True # cleaner row/col labels
)
# Map a scatter onto each facet using map_dataframe
g.map_dataframe(sns.scatterplot, x="Item_MRP", y="Item_Outlet_Sales", alpha=0.5)
# Labels, legend, title
g.set_axis_labels("Item_MRP", "Item_Outlet_Sales")
g.add_legend(title="Location Type")
g.fig.suptitle("Item_MRP vs Item_Outlet_Sales by Outlet Size and Location", y=1.03)
# Save the figure as an image
g.savefig("facetgrid_mrp_sales.png", dpi=150, bbox_inches="tight")
plt.show()
```

Item_MRP vs Item_Outlet_Sales by Outlet Size and Location



9. JointGrid (using JointGrid.plot_joint() and JointGrid.plot_marginals()):

- Create a JointGrid showing the joint distribution of Item_MRP and Item_Outlet_Sales, with marginal distributions plotted along the axes. Parameters to explore: kde=True for “scatterplot”.
- Save the figure as an image.

```
# Initialize the JointGrid
g = sns.JointGrid(data=clone_b_m, x="Item_MRP", y="Item_Outlet_Sales", height = 7)
#Map a scatter plot, joint the center
g.plot_joint(sns.scatterplot, alpha=0.5)
# marginal panels (top + right) histogram WITH the kde overlay
g.plot_marginals(sns.histplot, kde=True)

g.figure.suptitle("Joint distribution: Item_MRP vs Item_Outlet_Sales", y=1.02)

g.savefig("jointgrid_mrp_sales.png", dpi=150, bbox_inches="tight")
plt.show()
```

10. PairGrid (using PairGrid.map_diag() and PairGrid.map_offdiag()):

- Create a PairGrid showing histograms on the diagonal and scatterplots for the off-diagonal plots.
- Parameters to explore: color for both plots. Save the figure as an image.

```
g = sns.PairGrid(clone_b_m, vars=["Item_MRP", "Item_Outlet_Sales"], hue="Item_Fat_Content")
g.map_diag(sns.histplot, kde=True)
g.map_offdiag(sns.scatterplot, alpha=0.5)
g.add_legend()
g.savefig("PairGrid.png", dpi=150, bbox_inches="tight")
plt.show()
```

Additional Bivariate plots

```
g = sns.PairGrid(clone_b_m, vars=["Item_MRP", "Item_Outlet_Sales"])
g.map_diag(sns.histplot, kde=True)
g.map_lower(sns.kdeplot, cmap="Purples", fill=True) # bivariate KDE, cmap not palette
g.map_upper(sns.histplot) # bivariate histogram (note: histplot)
g.savefig("PairGrid.png", dpi=150, bbox_inches="tight")
plt.show()
```

